Low-Level Design (LLD) – Supply Chain Optimizer

(Pandas Data Analytics - 2 Class Architecture)

Difficulty Level: Intermediate | Total Marks: 20

Design Format: 2 Classes (Data + Optimizer) | 7 Independent Methods | 7 Tagged Test Cases

Summary of Design Requirements

Implement a supply chain optimization system using Pandas to analyze inventory and shipping performance.

The system uses two datasets:

- warehouse_stock.csv (SKU, Warehouse_ID, Stock, Safety, Cost)

- shipping_logs.csv (Order_ID, SKU, Warehouse_ID, Carrier, Order_Date, Delivery_Date, Status)

The system should identify stockout risks, analyze lead-time delays, and detect carrier performance bottlenecks.

Concepts Tested

- Pandas DataFrame Merging (Inner/Outer)

- Date arithmetic (Timedelta)

- GroupBy and Aggregation

- Percentage calculations and filtering

Class Architecture

1. Class "WarehouseManager" (Data Controller):

- Responsible for loading and merging CSV datasets.

- Handles the "State" of the raw data.

2. Class "SupplyOptimizer" (Logic Processor):

- Performs complex analytics on the DataFrames provided by the Manager.

- Calculates risks, rates, and totals.

Operations (Methods)

1. Load Data (Easy)

def load_data(self, stock_path: str, logs_path: str):

- Read CSVs into self.df_stock and self.df_logs.

2. Merge Inventory (Easy)

def get_merged_data(self) -> pd.DataFrame:

- Perform an inner join on "SKU" and "Warehouse_ID".

3. Identify Stockout Risk (Medium)

def get_at_risk_skus(self, df: pd.DataFrame) -> list:

- Filter where 'Stock' < 'Safety'. Return list of unique SKUs.

4. Calculate Mean Lead Time (Medium)

def calc_mean_lead_time(self, df: pd.DataFrame) -> float:

- Convert Date columns to datetime.
- Calc (Delivery_Date - Order_Date) in days.
- Return mean days (rounded 1f).

5. Inventory Holding Value (Medium)

def total_inventory_value(self, df_stock: pd.DataFrame) -> float:

- Sum of (Stock * Cost) for all rows.

6. Carrier Reliability (Medium)

def carrier_return_rate(self, df_logs: pd.DataFrame) -> dict:

- Return dict: {Carrier: % of 'Returned' status}.

7. Optimization Summary (Hard)

def generate_summary(self, df_stock, df_logs) -> dict:

- Return dict with keys: 'total_skus', 'critical_count', 'top_carrier'.

Test Cases & Marks Allocation:

Test Case ID	Description	Method	Marks
TC1	Load internal DataFrames	load_data()	0
TC2	Verify Inner Join merging	get_merged_data()	3
TC3	Filter sub-safety stock items	get_at_risk_skus()	3
TC4	Average transit days calculation	calc_mean_lead_time()	3
TC5	Aggregate total financial value	total_inventory_value()	3
TC6	Grouped return rate analysis	carrier_return_rate()	4
TC7	Multi-metric summary object	generate_summary()	4
Total			**20**

Visible Test Case Descriptions

TC1: DataFrames loaded and verified.

TC2: Merged DF contains expected columns.

TC3: identify SKUs where stock < safety.

TC4: Lead time mean matches expected float.

TC5: Financial value matches sum(stock*cost).

TC6: Dict contains carriers and their return percentages.

TC7: Summary dict provides high-level metrics.